CS 473 (Spring 2025)
Homework 2 (due Feb 13 Thu 10am)

Instructions: As in Homework 1.

Problem 2.1:

(a) Consider a function $C(\cdot,\cdot)$ defined by the following recursive formula: for all $i, j \geq 1$,

$$C(i,j) \;=\; \max\Big\{C(i-1,j-1)+f(i,j),\; C(i+2,j-2)+g(i,j),\; C(i+j,j)+h(i,j)\Big\},$$

with the base case $C(i,j) = 0$ if $i \leq 0$ or $j \leq 0$ or $i > n$. Here, f, g, h are given functions that can be evaluated in constant time.

(i) Write pseudocode to evaluate $C(n,n)$ using dynamic programming, and (ii) analyze the running time and space as a function of n.

(b) Consider a function $C(\cdot,\cdot)$ defined by the following recursive formula: for all $i, j \in \{1,\ldots,n-1\}$,

$$C(i,j) \;=\; \min_{k\in\{j+1,\ldots,n\}:\; g(i,j,k)\geq 0} \Big(C(i-1,k)+C(i+1,k)+f(i,j,k)\Big),$$

with base cases $C(i,j) = 0$ if $i = 0$ or $i = n$, and $C(i,j) = 0$ if $j = n$. Here, f, g are given functions that can be evaluated in constant time.

(i) Write pseudocode to evaluate $C(\lfloor n/2 \rfloor, 1)$ using dynamic programming, and (ii) analyze the running time and space as a function of n.

Solution:

(a) Pseudocode:

```
C = [[0] * (n+1)] * 3
for j in range(1, n+1):      # iterate j in increasing order
  roll C
  for i in range(n, 0, -1):  # iterate i in decreasing order
    term1 = C[1][i-1] if i-1>0 else 0
    term2 = C[0][i+2] if i+2<=n else 0
    term3 = C[2][i+j] if i+j<=n else 0
    C[2][i] = max(term1, term2, term3)
return C[2][n]
```

Time complexity is $O(n^2)$.

Space complexity is $O(n)$ using rolling array.

(b) Pseudocode:

```
C = [[0] * n for _ in range(n)]
root = n / 2
for j in range(n, 0, -1):
# evaluate from right to left
  for i in range(min(n-j, root), 0, -1):
    # evaluate from two sides to center
    C[root-i][j+i] = evaluate_min(C, root-i, j)
    C[root+i][j+i] = evaluate_min(C, root+i, j)
  C[root][j] = evaluate_min(C, root, j)
return C[root][1]
```

Iteration takes $O(n^2)$ steps, each step takes $O(n)$ time to evaluate minimum value, total time complexity is $O(n^3)$.
Space complexity is $O(n^2)$.

Problem 2.2: Recall the following formulation of the *line break* problem from class: given a sequence of positive integers $\langle a_1, \ldots, a_n \rangle$ and a number L, we want to split the sequence into contiguous subsequences $\langle a_1, \ldots, a_{i_1} \rangle$, $\langle a_{i_1+1}, \ldots, a_{i_2} \rangle$, $\ldots$, $\langle a_{i_{k-1}+1}, \ldots, a_n \rangle$, for some $0 = i_0 < i_1 < \cdots < i_{k-1} < i_k = n$, such that each subsequence sums to at most L, while minimizing the *penalty* $\sum_{j=1}^{k} f(L - a_{i_{j-1}+1} - \cdots - a_{i_j})$. Here, f is a given function that can be evaluated in constant time.

(a) Consider a variant of the problem in which we impose an additional constraint that each subsequence has at most 10 elements and the last subsequence has at most 5 elements (i.e., $i_j - i_{j-1} \leq 10$ for all $j = 1, \ldots, k-1$ and $i_k - i_{k-1} \leq 5$).

Describe a modified dynamic programming algorithm to solve this problem. Include the following steps: (i) define your subproblems and indicate how the final answer can be obtained, (ii) derive the recursive formula (including base cases) with brief justifications, (iii) specify a valid evaluation order, and (iv) analyze the running time and space (as a function of n). For this problem, you do not need to write pseudocode if your recursive formula and evaluation order are described clearly. And you do not need to write pseudocode to output the optimal solution; just the optimal value suffices.

(b) Consider a variant of the problem in which we are given an additional input parameter m and we impose an additional constraint that the number of subsequences used (denoted k above) is exactly m. (But we don't impose the constraint from part (a).)

Describe a modified dynamic programming algorithm to solve this problem. Include the following steps: (i) define your subproblems, (ii) derive the recursive formula (including base cases) with brief justifications, (iii) specify a valid evaluation order, and (iv) analyze the running time and space (as a function of n and m). For this problem, you do not need to write pseudocode if your recursive formula and evaluation order are described clearly. And you do not need to write pseudocode to output the optimal solution; just the optimal value suffices.

Solution:

(a) (i) Subproblems:
For each $i = 1, 2, \ldots, n$, define,

$$P(i) = \text{minimum penalty for subsequence } \langle a_i, \ldots, a_n \rangle.$$

Want $P(0)$.

(ii) Recursive formula:

$$P(i) = \min_{\substack{j \in (i+1, \ldots, n) \\ i_j - i_{j-1} \leq 10 \text{ for all } j = 1, \ldots, k-1 \\ i_k - i_{k-1} \leq 5 \\ \alpha < L}} (P(j) + f(L - \alpha)),$$

where $\alpha = \sum_{m=i+1}^{j} a_m$. Base case is $P(n+1) = 0$.

(iii) Evaluation order:
Pre-compute $S(i) = \sum_{m=i+1}^{n} a_m$, then evaluate $P(i)$ from n to 1.

(iv) Time and space analysis:
Iteration takes $O(n)$ steps, each step evaluates $P(j)$ and $f(L-\alpha)$ in $O(1)$ time with pre-computed $S(i)$, total time complexity is $O(n)$.
$P(i)$ and $S(i)$ both take $O(n)$ space, total space complexity is $O(n)$.

(b) (i) Subproblems:
For each $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n$, define,

$$P(i, j) = \text{minimum penalty for the } i\text{th subsequence ending at } a_j.$$

Want $P(m, n)$.

(ii) Recursive formula:

$$P(i, j) = \min_{\substack{u \in (1,2,\ldots,j) \\ \alpha < L}} (P(i-1, u) + f(L-\alpha)),$$

where $\alpha = \sum_{m=u+1}^{j} a_m$. Base case is $P(0, 0) = 0$, $P(0, j) = +\infty$ for $j = 1, 2, \ldots, n$.

(iii) Evaluation order:
Pre-compute $S(i) = \sum_{v=i+1}^{n} a_v$, then evaluate i from 1 to m, with each i, evaluate j from 1 to n.

(iv) Time and space analysis:
Iteration takes $O(mn)$ steps, each step evaluates minimum for $O(n)$ times, each evaluation takes $O(1)$ time with pre-computed $S(i)$, total time complexity is $O(mn^2)$.
$P(i, j)$ takes $O(mn)$ space (or $O(n)$ space by using rolling array), $S(i)$ takes $O(n)$ space, total space complexity is $O(n)$.

Problem 2.3: We have an ordered list of n customers, all arriving at time 0 (i.e., the time when the store opens). We have 3 servers. For each $i = 1, \ldots, n$, customer i requires ℓ_i minutes to serve, where ℓ_i is a positive integer at most L. We want to assign customers to servers, so as to minimize the sum of the waiting times of the customers. The customers need to be served in the given order, and *all* customers must be served. Describe a dynamic programming algorithm to solve this problem in time polynomial in n and L.

Example: for $\ell_1 = 5$, $\ell_2 = 3$, $\ell_3 = 10$, $\ell_4 = 20$, $\ell_5 = 6$, $\ell_6 = 15$, and $L = 20$, one (not necessarily optimal) solution is to assign customers 1,3 to server 1, and customers 2,4,5 to server 2, and customer 6 to server 3. Then customer 3 has to wait 5 minutes, customer 4 has to wait 3 minutes, and customer 5 has to wait $3 + 20 = 23$ minutes. The total waiting time is $5 + 3 + 23 = 31$ minutes.

Include *all* of the following steps: (i) first define your subproblems precisely, (ii) then derive the recursive formula (including base cases) with brief justifications, (iii) specify the evaluation order, (iv) write pseudocode to output the optimal value (i.e., the minimum total waiting time), (v) write pseudocode to output the optimal solution (i.e., the assignment of customers to servers), and (vi) analyze the running time and space as a function of n and L.

[Hint: define a class of subproblems with 4 parameters (the number of subproblems may be a function of both n and L)...]

Basic Solution:

(i) Subproblems:
For each $i = 1, \ldots, n$, define

$$P(i, s_{1_i}, s_{2_i}) = \text{minimum total waiting time when each server's serving time is } s_{1_i}, s_{2_i}, s_{3_i} \text{ after customer } i,$$

subject to

$$s_{1_i} + s_{2_i} + s_{3_i} = \sum_{k=0}^{i} \ell_k.$$

Want

$$\min_{s_1, s_2 \in (0, \ldots, nL)} P(n, s_1, s_2).$$

(ii) Recursive formula:
For all $i = 1, \ldots, n$,

$$P(i, s_{1_i} + \ell_i, s_{2_i}) = \min\{P(i, s_{1_i} + \ell_i, s_{2_i}), P(i-1, s_{1_i}, s_{2_i}) + s_{1_i}\},$$
$$P(i, s_{1_i}, s_{2_i} + \ell_i) = \min\{P(i, s_{1_i}, s_{2_i} + \ell_i), P(i-1, s_{1_i}, s_{2_i}) + s_{2_i}\},$$
$$P(i, s_{1_i}, s_{2_i}) = \min\{P(i, s_{1_i}, s_{2_i}), P(i-1, s_{1_i}, s_{2_i}) + s_{3_i}\},$$

where

$$s_{3_i} = \sum_{k=0}^{i} \ell_k - s_{1_i} - s_{2_i}.$$

Base case is $P(0, 0, 0) = 0$.

(iii) Evaluation order:
Evaluate i from 1 to n, evaluate s_1, s_2 from 0 to nL, for each (i, s_1, s_2), evaluate $P(i, s_{1_i} + \ell_i, s_{2_i})$, $P(i, s_{1_i}, s_{2_i} + \ell_i)$ and $P(i, s_{1_i}, s_{2_i})$ iff $(i-1, s_{1_i}, s_{2_i})$ is reachable.

(iv) Optimal value:

```
L = pre-computed sum(\ell_i)
P = array of size (2, nL, nL) initiated with +inf
S = array of size (n+1, nL, nL) of previous state (s1, s2)
P[0][0][0] = 0
for i = 1 to n:
  roll P
  for s1 = 0 to n*L:
    for s2 = 0 to n*L:
      if P[i-1][s1][s2] != +inf:  # previous state is reachable
        evaluate P according to step (ii)
        update S
return minmum value of P[n]
       optimal state if optimal solution is needed
```

(v) Optimal solution:

```
state = optimal state of n (s1_n, s2_n)
assignment = [0]*n
for i = n to 1:
  previous_state = S[k_i][s1_i][s2_i]
  # get assignment by comparing server time
  assignment[i] = k of any sk whose serve time is different
                  between the states, k=3 otherwise
  state = previous_state
return assignment
```

(vi) Time and space analysis:
Iteration takes $O(n \times (nL)^2)$ steps, each step takes $O(1)$ time to evaluate P and record state and assignment S, optimal solution takes extra $O(n)$ time, total time complexity is $O(n^3L^2)$.
P takes $O((nL)^2)$ space using rolling array, S takes $O(n \times (nL)^2)$ space, total space complexity is $O(n^3L^2)$. If optimal solution is not needed, total space complexity is $O(n^2L^2)$.

Bonus Solution:

Idea: Only record possible states using hash table.

(i&ii) Same.

(iii) Evaluation order:
Evaluate i from 1 to n, for each possible previous states of $(i-1, s_1, s_2)$, evaluate (i, s_1, s_2) according to step (ii).

(iv) Optimal value:

```
L = pre-computed sum(\ell_i)
P_pre = hash table {state(s1, s2): total waiting time}
        of previous customer
P_cur = hash table {state(s1, s2): total waiting time}
        of current customer
S = hash table {state(i, s1, s2):
    previous state(i-1, s1_prev, s2_prev)}
P_cur[(0, 0)] = 0
for i = 1 to n:
  P_pre = P_cur, clear P_cur
  for each state in keys of P_pre:
    evaluate P according to step (ii), update P_cur
    update S
return minimum value in values of P_cur,
       optimal state if optimal solution is needed
```

(v) Optimal solution:

```
state = optimal state of n (s1_n, s2_n)
assignment = [0]*n
for i = n to 1:
  previous_state = S[(i, s1_i, s2_i)]
  # get assignment by comparing server time
  assignment[i] = k of any sk whose serve time is different
                  between the states, k=3 otherwise
  state = previous_state
return assignment
```

(vi) Time and space analysis:

Iteration takes $O(n)$ steps, each step evaluates $O(n^2)$??? states, $O(1)$ time using hash table,